GUIDE DE COMPRÉHENSION

AI Claims Agent

Le projet expliqué pas à pas, sans prérequis

Ce guide explique le trajet d'un dossier, le rôle de n8n et de chaque agent, le RAG depuis les bases, les modèles configurés, le calcul du risque et de la confiance, les fichiers du dépôt et la correspondance avec le cours et l'Advanced DL Exam.

En une phrase
React reçoit le dossier, Django l'enregistre, n8n coordonne les appels, Document/Vision/Policy examinent les pièces en parallèle, puis Fraud et Decision combinent leurs résultats.

Sommaire

1. Vue générale et workflow
2. Rôle de chaque agent et choix des modèles
3. RAG expliqué depuis zéro
4. Multimodalité et fusion
5. Calcul du risque, de la décision et de la confiance
6. Carte des fichiers du projet
7. Comparaison avec le cours
8. Advanced DL Exam : exigences et correspondance
9. Limites, glossaire et questions de soutenance

Vérifié le 28 octobre 2026 à partir du code, de la configuration locale, de l'énoncé d'examen et des supports de cours fournis.

1. Vue générale et workflow

Le système reçoit une déclaration automobile avec des pièces possibles : formulaire, constat PDF, facture, photos et éventuellement audio. Il extrait les faits, observe les dégâts, cherche les clauses, compare les éléments, puis recommande une suite.

Dépôt	Enregistrement	Orchestration
React: formulaire + pièces	Django crée dossier et fichiers	n8n reçoit le webhook
↓ les trois spécialistes travaillent en parallèle		
Document	**Vision**	**Policy / RAG**
PDF / OCR / audio -> faits	Photos -> zones visibles	Question -> clauses du contrat
↓ les résultats sont réunis		
Fraud	**Decision**	**Retour et affichage**
Contrôle des incohérences	ACCEPT / INVESTIGATE / REJECT	Callback Django -> React

Le workflow exporté décrit précisément l'ordre des appels. Le tableau ci-dessous explique ce qui se passe dans chaque étape.

Étape	Composant	Ce qui se passe
1. Dépôt	React	Formulaire du sinistre et ajout des pièces.
2. Enregistrement	Django REST API	Valide les champs, crée Claim et ClaimDocument, enregistre les fichiers et prépare une timeline.
3. Déclenchement	Django vers n8n	Webhook avec identifiant du dossier, URL interne et URL de retour.
4. Analyse parallèle	Document, Vision, Policy/RAG	n8n appelle un endpoint Django; Django lance ces fonctions en trois threads. Elles travaillent sur des entrées différentes.
5. Contrôle croisé	Fraud	Compare les faits extraits, les zones visuelles et les pièces manquantes.
6. Recommandation	Decision	Applique des règles dans un ordre défini.
7. Retour	n8n vers Django	Callback protégé par un jeton; Django conserve décision, raisons, références, sorties et durées.
8. Affichage	React	Fiche dossier et page Traçabilité montrent résultats, fichiers et timeline.

Pourquoi le Document Agent peut sembler lent
Il peut traiter plusieurs PDF par pypdf et Tesseract, transcrire l'audio avec Faster-Whisper, puis demander à Qwen de structurer les faits. Les trois spécialistes sont lancés en parallèle, mais Ollama est verrouillé pour éviter de charger plusieurs modèles lourds en même temps. Les estimations de temps restant affichées dans l'interface sont approximatives.

2. Rôle des agents et choix des modèles

Les noms suivants viennent du fichier .env actuellement présent. Le serveur renvoie aussi, dossier par dossier, le moteur et le modèle réellement utilisés. Si Ollama échoue, certains agents passent en mode fallback. Les valeurs par défaut de settings.py peuvent différer des valeurs définies dans .env.

Agent	Entrée et travail	Sortie	Modèle configuré
Document	PDF, formulaire, facture, constat et transcription. pypdf lit le texte natif; Tesseract OCR reconnaît les scans; les deux résultats sont comparés. Faster-Whisper transcrit les WAV français. Qwen transforme le texte en champs structurés.	Date, lieu, faits, numéro de police, parties, montants trouvés, pièces manquantes, comparaison pypdf/OCR.	qwen2.5:3b; faster-whisper small pour audio
Vision	Jusqu'à six photos transmises comme images à Ollama. Le prompt limite les zones visibles à rear/front/left/right et dit de ne pas deviner.	Dégâts visibles, zones, sévérité et qualité; confiance VLM non calibrée.	llava:7b
Policy/RAG	Récit, date, lieu, clauses retrouvées et, s'il existe, Conditions particulières dont le numéro de police correspond.	covered, excluded ou insufficient_information, explication et références pages.	qwen2.5:3b + no mic-embed-text:latest
Fraud	Compare la zone mentionnée dans le récit avec la zone fournie par Vision; ajoute les points de règles déclenchées.	Incohérences documentées, score et état complet/incomplet. Ne prouve pas une fraude juridique.	Aucun LLM; Python
Decision	Reçoit couverture, risque et complétude; suit des règles ordonnées.	ACCEPT, INVESTIGATE ou REJECT, motifs et confiance heuristique.	Aucun LLM; Python

Pourquoi ces choix ?

Qwen 2.5 3B : modèle de texte local de taille modérée, configuré pour extraire des faits et lire les passages de contrat avec une sortie JSON. Il limite le besoin de service distant et convient à une démonstration locale; il reste moins robuste qu'un modèle plus grand sur les cas ambigus.

LLaVA 7B : modèle vision-langage capable de recevoir la photo et une instruction textuelle dans le même appel. Il peut décrire une zone endommagée directement; ses erreurs possibles sont contrôlées et les contradictions marquent l'analyse incomplète.

Nomic embeddings : modèle spécialisé pour encoder des textes à des fins de recherche par proximité; il ne rédige pas la réponse. Faster-Whisper convertit de la parole en texte, il n'est pas un LLM de chat.

Les deux agents finaux
Fraud et Decision utilisent des règles explicites plutôt qu'un LLM. Cela rend leurs scores et priorités reproductibles : mêmes faits en entrée, même résultat selon le code.

3. Le RAG expliqué depuis zéro

RAG veut dire Retrieval-Augmented Generation : récupération de passages, enrichissement du contexte, génération d'une réponse. Un LLM seul ne connaît pas forcément le contrat exact de ce dossier. Le RAG recherche d'abord des clauses dans un corpus choisi, puis transmet seulement ces clauses au LLM. Le modèle ne relit pas tout le PDF à chaque demande.

1. Le PDF	2. Les morceaux	3. Les vecteurs	4. La recherche	5. Réponse
Conditions générales AXA	500 mots par fenêtre; 50 mots répétés entre deux fenêtres	Nomic convertit chaque passage en liste de nombres	Cosinus entre question et passage; tri des 5 meilleurs	Qwen explique avec les passages et les Conditions particulières disponibles

Le corpus exact

Le fichier backend/claims/corpus/axa_mon_auto_septembre_2026.pdf contient les Conditions générales publiques AXA France « Mon Auto », version septembre 2026 : 104 pages au total. Le manifeste backend/claims/policy_corpus.json décrit le pays, la compagnie, la date, l'URL source et les pages indexées, pages 5 à 103. Les Conditions particulières ne font pas partie de ce corpus général : dans les dossiers de démonstration, elles sont des fichiers synthétiques fournis en pièce jointe.

Pourquoi découper en morceaux ?

Un très long PDF dépasse la quantité de texte qu'un modèle peut traiter efficacement dans une requête. Un morceau court permet aussi de remonter l'article utile au lieu d'ajouter tout le contrat. Le code découpe chaque page en fenêtres de 500 mots avec 50 mots de chevauchement. Les métadonnées conservent page, section estimée, version et source. Le chevauchement répète une petite partie du texte pour garder le contexte près d'une frontière de fenêtre.

Comment fonctionne la recherche sémantique ?

Un modèle d'embeddings transforme un texte en vecteur, c'est-à-dire une liste de nombres représentant son sens approximatif. Le système encode les morceaux du contrat et la question du sinistre. Il compare ensuite les directions des vecteurs avec la similarité cosinus :

$$\text{cosinus}(q, d) = (q \cdot d) / (||q|| \times ||d||)$$

Un score proche de 1 signifie que les vecteurs pointent dans des directions proches. Un score proche de 0 signifie qu'ils se ressemblent peu dans cet espace. Ce score sert à classer les passages; il ne signifie pas que le contrat couvre le sinistre. Le code garde les cinq meilleurs passages, c'est top-k avec k=5. Les vecteurs sont actuellement gardés en cache mémoire Python, pas dans une base vectorielle persistante.

Exemple de bout en bout

Question : « Le choc arrière accidentel est-il couvert ? » La requête inclut le récit, la date et le lieu plus des termes de recherche tels que collision, dommages au véhicule et exclusions. Nomic code la requête et les fenêtres du contrat en vecteurs. Le retriever choisit les cinq fenêtres les plus proches. Qwen reçoit ces fenêtres et les Conditions particulières si leur texte correspond au numéro de police. Il retourne un statut, des motifs, des références de page et les informations manquantes dans un schéma JSON validé.

Règle d'abstention
Si les Conditions particulières du bon contrat ne sont pas disponibles, le système ne doit pas confirmer la garantie : il renvoie insufficient_information. Le corpus général montre quelles garanties existent, pas lesquelles cette personne a achetées.

3.1 Résultats du test RAG et limites

Le dépôt teste 16 questions dont les pages pertinentes ont été annotées manuellement. Deux stratégies sont comparées : page entière ou fenêtres fixes. Deux classements sont évalués : comptage lexical des mots et embeddings Nomic avec cosinus.

Méthode	Segmentation	Passages	Hit@5	MRR@5
Lexicale	pages	99	0,812 (13/16)	0,658
Lexicale	500 mots / 50 chevauchement	131	0,812 (13/16)	0,661
Nomic + cosinus	pages	99	0,938 (15/16)	0,604
Nomic + cosinus	500 mots / 50 chevauchement	131	0,938 (15/16)	0,596

Hit@5 : au moins une page attendue apparaît dans les cinq premiers passages. MRR@5 : rang moyen réciproque du premier passage pertinent; rang 1 vaut 1, rang 2 vaut 0,5, passage absent vaut 0.

Les embeddings ont trouvé une page pertinente dans 15 questions sur 16; le lexical dans 13 sur 16. Le découpage en fenêtres n’améliore pas clairement le classement dans ce petit test. Il est gardé surtout pour maîtriser la longueur envoyée au modèle. Les vecteurs sont recalculés et ne sont pas stockés durablement. Une seule compagnie et une seule annotatrice ont produit ce jeu; il ne mesure pas la justesse de la décision finale.

Limites importantes

• Le corpus est AXA France seulement; il ne justifie pas la couverture d’un contrat tunisien ou d’un autre assureur.

• Les 16 requêtes sont peu nombreuses et leurs pages ont été annotées par une personne.

• Pas de reranker ni de base vectorielle persistante; le fallback lexical ne confirme pas la couverture.

• Une extraction erronée ou une mauvaise interprétation de Qwen peut produire une référence inadéquate; ouvrir la page contractuelle reste nécessaire.

4. Multimodalité et fusion

Le projet traite séparément plusieurs modalités. pypdf lit le texte natif des PDFs; PyMuPDF rend les pages puis Tesseract OCR lit les scans; LLaVA analyse les photos; Faster-Whisper transcrit l'audio. Les sorties textuelles sont ensuite rapprochées par les agents Fraud et Decision. Cela s'appelle une fusion tardive, ou late fusion.

Type de fusion	Principe	Dans le projet
Précoce (early)	Assembler les entrées multimodales avant l'analyse.	Non utilisée.
Intermédiaire	Fusionner des représentations internes/embeddings multimodaux au milieu des modèles.	Non utilisée; les modèles ne partagent pas leurs couches internes.
Tardive (late)	Analyser chaque type séparément puis combiner les résultats.	Oui: Document fournit les faits, Vision fournit les zones, Policy les clauses; les règles les rapprochent.

Pourquoi pas CLIP ou CLAP ?

CLIP rapproche images et textes pour les comparer ou les retrouver; CLAP fait un rapprochement analogue pour l'audio et le texte. Ici, le besoin est de décrire les dommages visibles et de transcrire la voix. LLaVA répond directement à une consigne portant sur des images et Faster-Whisper répond au besoin de transcription. CLIP/CLAP ne sont pas utilisés dans le code actuel.

Comparaison pypdf / OCR

Pour chaque PDF de dossier, le système garde le texte extrait par pypdf et celui reconnu par Tesseract, calcule leur similarité et des segments communs, puis choisit un texte : pypdf s'il contient au moins 80 caractères, sinon l'OCR s'il est plus long. La comparaison apparaît dans les résultats Document et la page Traçabilité. Ce seuil aide à choisir entre texte natif et scan; il ne garantit pas que les mots reconnus sont corrects.

5. Calcul du risque, de la décision et de la confiance

Les deux nombres affichés répondent à des questions différentes. Le risque ajoute les points des règles de cohérence présentes. La confiance est une formule de pénalités liée à la complétude et au risque. Elle n'est pas une probabilité calibrée.

Score de risque

Règle active	Points	Interprétation
La zone du récit et celle de Vision se contredisent	+50	Signal à vérifier; ne prouve pas à lui seul une fraude.
Pièce requise manquante	+25	Le dossier est incomplet.
Total	min(100, somme des points)	Plafond à 100; si vision incomplète, score final = null.

Exemples : aucune règle = 0; une contradiction = 50; contradiction et pièce manquante = 75. Si les photos ne sont pas analysées, Fraud renvoie « Non évalué » (null), et non 0. Il peut garder un score partiel observé à titre de diagnostic.

Ordre des décisions

Ordre	Condition	Recommandation
1	Exclusion explicite liée au cas	REJECT
2	Couverture inconnue ou Fraud incomplet	INVESTIGATE
3	Risque positif	INVESTIGATE
4	Couverture confirmée, analyse complète, risque nul	ACCEPT

Calcul de confiance

La base est 0,95. Le code soustrait les pénalités suivantes, additionne les pertes, puis borne le résultat entre 0,45 et 0,95 :

Événement	Pénalité
Pièce manquante	0,18
Aucune photo exploitable	0,12
Couverture inconnue	0,20
Vision incomplète quand des dégâts sont signalés	0,12
Risque positif	risque / 100 × 0,25

Formule : confiance = max(0,45; min(0,95; 0,95 - somme des pénalités)).

Exemple 1 : couverture confirmée, vision complète, aucune anomalie : 0,95 - 0 = 0,95 (95 %), décision ACCEPT. Exemple 2 : seule la garantie est inconnue : 0,95 - 0,20 = 0,75 (75 %), mais décision INVESTIGATE. Exemple 3 : une contradiction donne risque 50, donc pénalité risque = 50/100 × 0,25 = 0,125; la confiance devient 0,825 (82,5 %) si aucune autre pénalité s'applique; décision INVESTIGATE.

À retenir
0/100 ne signifie pas « sans risque » ni « contrat couvert ». Cela signifie seulement qu'aucune des règles de risque actuelles n'a ajouté de points. La décision tient compte séparément de la couverture et de l'état des analyses.

6. Carte des fichiers du dépôt

Voici les principaux fichiers source et livrables. Les caches, bibliothèques installées, fichiers uploadés par les utilisateurs et fichiers temporaires ne sont pas tous listés individuellement.

Fichier / dossier	Contenu et rôle
README.md	Démarrage, architecture, modèles, corpus et configuration de sécurité.
docker-compose.yml	Lance PostgreSQL, backend Django, frontend React/Vite et n8n; relie leurs réseaux et volumes.
.env.example	Exemple de configuration. Le fichier .env privé contient les valeurs machine et n'est pas recopié ici.
Advanced DL-Exam.pdf	Énoncé : conditions du projet final, livrables, barème et calendrier.
backend/config/settings.py	Django, base, sécurité, CORS, Ollama, Whisper, email.
backend/config/urls.py	Routeur API racine.
backend/config/wsgi.py et __init__.py	Point d'entrée WSGI et marqueur de package Python.
backend/manage.py	Commandes de gestion Django, migrations, serveur et tests.
backend/claims/apps.py, admin.py, __init__.py	Configuration de l'application, intégration admin et marqueur de package.
backend/claims/models.py	Schéma base de Claim, ClaimDocument, InsuranceContact, statuts et décisions.
backend/claims/serializers.py	Validation et conversion JSON/fichiers pour l'API.
backend/claims/views.py	Endpoints : création/relance, agents, pièces, contacts, callback, timeline.
backend/claims/urls.py	Associe les URL aux endpoints.
backend/claims/agents.py	Extraction PDFs/OCR/audio; appels Ollama; RAG, Vision, Fraud, Decision et calculs.
backend/claims/services.py	Webhook vers n8n et transmission email à un responsable.
backend/claims/tests.py	Tests unitaires/API pour agents, règles, documents et RAG.
backend/claims/policy_corpus.json	Source, version, pays et pages du corpus de contrat.
backend/claims/corpus/axa_mon_auto_septembre_2026.pdf	Conditions générales publiques AXA France utilisées comme base RAG.
backend/claims/migrations/0001-0003	Création du schéma, ajout des contacts/transmissions et statut email.
frontend/src/App.jsx	Pages React : dossiers, dépôt, fiche, pièces, traçabilité, timeline, contacts.
frontend/src/api.js	Appels HTTP du navigateur à l'API Django.
frontend/src/main.jsx	Point d'entrée React.
frontend/src/styles.css	Présentation graphique.
frontend/package.json	Dépendances et commandes Vite.
frontend/index.html	Page HTML racine de l'application.
frontend/Dockerfile	Image de développement du frontend.
n8n/ai-claims-workflow.json	Workflow exporté : webhook, spécialistes, Fraud, Decision, callback.
output/documents/bibliotheque_prompts.md	Rôle et méthode des prompts, échecs observés, corrections et limites.
output/documents/plan_demo_15min.md	Déroulé de la démonstration et questions possibles.
output/evaluation/evaluate_rag.py	Script de comparaison des segmentations et méthodes de recherche.
output/evaluation/evaluation_rag.md	Questions, pages attendues, résultats Hit@5/MRR@5 et limites.
output/latex/.../main.tex	Source du rapport technique universitaire de 8 pages.
output/pdf/dossier-demo-accepte-france/	Pièces synthétiques du scénario d'acceptation.
output/pdf/dossier-demo-refus-france/	Pièces synthétiques du scénario d'exclusion/refus.
output/images/ et docs/	Diagrammes de workflow et architectures logique/physique.

Parcours dans le code

React envoie le formulaire à /api/claims/. ClaimSerializer crée les objets pièces; ClaimViewSet.perform_create appelle trigger_n8n dans services.py. Le nœud n8n appelle les endpoints internes de views.py. _execute_agent enregistre état de progression; complete_claim accepte le retour avec X-Callback-Token. React lit agent_results pour construire la page Traçabilité.

7. Correspondance avec le cours

Le code applique certaines notions du cours; il n'implémente pas tous les modèles ou algorithmes présentés dans les supports.

Support	Concepts du cours	Dans ce projet	Ce qui n'est pas fait
ch5_transformers.pdf	Attention et architecture Transformer.	Appels à des modèles préentraînés Qwen/LLaVA, eux-mêmes modèles de la famille Transformer/VLM.	Pas d'attention codée à la main ni d'entraînement depuis zéro.
LLM.pdf	Cycle de choix modèle, ressources matérielles, prompting et paramètres.	Petits modèles locaux; température 0 et JSON pour plus de régularité.	Pas de benchmark comparatif exhaustif de fournisseurs.
RAG.pdf	Corpus, chunking, embedding, retrieval, contexte puis génération.	Corpus AXA, fenêtre 500/50, embedding Nomic, cosinus top 5, Qwen. Évaluation 16 requêtes.	Pas de store vectoriel persistant ni reranker.
Multimodality.pdf	CLIP, BLIP, Flamingo, LLaVA, audio et fusion.	LLaVA pour photos; Tesseract pour scan; Whisper pour parole; fusion tardive.	Pas de CLIP/CLAP ni de fusion intermédiaire apprise.
Agentic AI.pdf	Agent vs workflow, outils, mémoire, planification, orchestration et observabilité.	n8n orchestre un workflow fixé; Django expose les agents et garde le résultat/timeline.	Pas d'agent autonome qui choisit librement sa prochaine action; Fraud/Decision sont Python.
ch7_fine_tuning.pdf	Le support aborde prompting et réglages LLM, ainsi que fine-tuning.	Prompts versionnés et réglages; modèles préentraînés employés sans fine-tuning.	Aucun fine-tuning, faute de corpus annoté adapté et parce que le prototype n'en a pas besoin.

Lien entre les notions
Le prompting dit comment répondre; le RAG apporte les clauses utiles; la multimodalité lit d'autres types de pièces; n8n organise les appels; les règles Python relient les résultats et rendent la recommandation compréhensible.

8. Advanced DL Exam : l'énoncé et le projet

L'énoncé s'intitule « Advanced Deep Learning - Final Project: AI Automation ». Il demande de concevoir et construire une automatisation IA fonctionnelle sur un sujet libre et utile, en utilisant n8n, Zapier, Make ou équivalent comme colonne vertébrale. Le thème du projet n'est pas noté en soi; ce sont les choix d'ingénierie qui comptent.

Exigences obligatoires et réponse du projet

Exigence de l'assignment	Ce qu'elle veut dire	Éléments du projet
1. Automation platform	Pipeline déclenchable de bout en bout, pas des cellules lancées à la main.	Webhook n8n lancé par Django; spécialistes, Fraud, Decision, callback. Le graphe d'erreurs/reprises peut être renforcé.
2. Prompt engineering	Bibliothèque versionnée pour chaque prompt : rôle, technique, échecs, correctifs; avant/après pour les prompts clés.	output/documents/bibliotheque_prompts.md contient les prompts Document/Vision/Policy et les itérations d'échec/correction.
3. RAG	Justifier corpus, source, taille/format; chunking, longueur/chevauchement; embedding/store; top-k, filtres/reranking; évaluation courte. Comparer 2 chunkings est fortement apprécié.	Contrat public AXA, Nomic, fenêtres 500/50, top-5 cosinus; pas de base vectorielle persistante ni reranker; comparaison pages/fenêtres et 16 questions.
4. Multi-agent	Plusieurs agents; justifier architecture et alternative; décrire responsabilité, outils, circulation des données.	Master/worker hybride : les trois spécialistes sont indépendants et tournent en parallèle; Fraud attend leurs sorties puis Decision attend Fraud. Tout faire en séquence serait plus lent; un appel monolithique serait moins traçable.
5. Multimodality	Au moins une modalité non textuelle; expliquer comment elle entre dans le pipeline et ce qu'elle apporte.	Photos via LLaVA, PDF via pypdf/OCR, audio facultatif via Whisper.

Livrables spécifiés

Les livrables doivent être remis la veille de la séance finale.

Livrable demandé	Correspondance
Workflow exporté (JSON par exemple) ou dépôt avec instructions.	Dépôt du projet, README.md, n8n/ai-claims-workflow.json.
Rapport technique de 8 pages maximum couvrant les cinq points.	Source LaTeX et PDF rapport technique de 8 pages.
Prompt library séparée.	output/documents/bibliotheque_prompts.md.
Courte évaluation retrieval: questions, résultats, commentaires.	output/evaluation/evaluation_rag.md et script Python associé.

Calendrier, soutenance et notation

Élément de l'énoncé	Contenu
Organisation	Équipes de 5 à 6 étudiants; 5 semaines; présence et courte mise à jour obligatoires au point de suivi hebdomadaire.
Soutenance	15 minutes de démonstration et 10 minutes de questions. Chaque membre parle et peut répondre sur toutes les parties.
Prompt engineering	20 %
RAG	25 %
Architecture multi-agent	25 %
Multimodalité	15 %
Démo, rapport et Q&A;	15 %
Conseil de l'énoncé	Justification avant complexité; un système simple et défendable vaut mieux qu'un système compliqué; signaler les échecs et ce qui n'a pas marché. Les modèles locaux sont autorisés; surveiller le coût des API.

8.1 Jalons des cinq semaines

Semaine	Attendu écrit dans l'énoncé	Correspondance au dépôt
1	Définir cas d'usage, rôles, plateforme, première architecture.	Cadrage sinistres; choix React/Django/n8n; diagrammes.
2	Collecter le corpus, implémenter le chunking, premiers essais retrieval.	PDF AXA versionné; fenêtres; test RAG.
3	Agents communiquant, prompts v1, multimodal isolé.	Endpoints d'agents, Ollama, OCR/photo/audio, prompt library.
4	Intégration bout à bout, prompts ajustés selon les échecs, gestion d'erreurs.	Workflow n8n, étapes spécialisées et cas de correction documentés.
5	Tests, rapport et répétition démo.	Rapport 8 pages, évaluation, fichiers de démonstration et répétition à préparer.

9. Questions utiles pour la soutenance

Question	Réponse courte conforme au code
Pourquoi n8n ?	Il rend l'ordre des étapes visible et déclenche les appels. Django conserve données et règles.
Est-ce un système multi-agent autonome ?	C'est une architecture à responsabilités spécialisées et workflow fixé; elle ne planifie pas librement ses outils.
Pourquoi « À examiner » avec risque 0 ?	Risque 0 signifie qu'aucune règle de risque ne s'est déclenchée. Couverture inconnue suffit à demander un examen.
La confiance 95 % est-elle une probabilité ?	Non, c'est une base heuristique moins des pénalités; elle n'est pas calibrée par statistique.
Qu'est-ce qui manque avant un usage réel ?	Jeu représentatif validé par experts, plusieurs contrats/assureurs, contrôle des erreurs, authentification/permissions, sécurité et validation juridique.

Références des documents et fichiers consultés

Énoncé : Advanced DL-Exam.pdf fourni dans le dépôt. Supports : Agentic AI.pdf, RAG.pdf, Multimodality.pdf, LLM.pdf, ch5_transformers.pdf et ch7_fine_tuning.pdf dans D:/5ds/advanced. Code : README.md; backend/claims/agents.py, models.py, serializers.py, views.py, services.py, policy_corpus.json; backend/config/settings.py; frontend/src/App.jsx et api.js; n8n/ai-claims-workflow.json. Évaluation et prompts : output/evaluation/evaluation_rag.md et output/documents/bibliotheque_prompts.md.

Note sur les données
Le PDF des Conditions générales AXA est une source publique réelle. Les dossiers et Conditions particulières fournis pour la démonstration sont synthétiques; les scores et règles décrivent un prototype, pas un outil assurantiel certifié.